Lab 6 - 1B

Jibon Naher[1] and Samnun Azfar[2]

[1]Lecturer, CSE
[2]Junior Lecturer,CSE

February 26, 2026

General Instructions

1. Use the PC from the lab.
2. Submit your mobile at the front.
3. Turn off your internet connection.
4. Your time is 50 minutes.

Part A : Sequential Launch of Space Systems

Problem Statement
In a space mission control system, three subsystems — Fuel System, Navigation System, and Launch System — must activate in a strict order for safety and synchronization.

Points to note

- Even though these systems (threads) start in random order, they must always initialize as: Fuel -> Navigation -> Launch
- Create three threads for fuel, navigation and launch.
- Use two semaphores: sem_nav -> Signals that the fuel system is ready, sem_launch -> Signals that navigation is ready.

Expected Output

```
[Navigation Thread] Waiting for fuel system...
[Launch Thread] Waiting for navigation system...
[Fuel Thread] Fuel system activated. Signal sent.
[Navigation Thread] Navigation system online. Signal sent.
[Launch Thread] Launch sequence initiated!
```

PART B: Readers–Writers Synchronization Problem

In a database system, multiple processes may either read or write shared data. To maintain data consistency, the following conditions must hold: Multiple readers can read the shared data simultaneously, Only one writer can modify the data at a time, When a writer is writing, no reader may access the data.

Your task is to simulate this behavior using threads and semaphores to ensure proper synchronization between readers and writers.

Input Description

- Create multiple reader and writer threads that access a shared variable (e.g., data). Readers should display the current value of data. Writers should increment or update the shared variable.
- All threads start in random order, but synchronization must ensure that: Multiple readers can read concurrently and Only one writer can write at a time.

Hints

- Use an integer variable read_count initialized to 0.
- Use two semaphores: mutex -> to protect the read_count variable, write_block -> to block writers when readers are active, and vice versa.

Expected Output

```
[Reader 1] Waiting to read...
[Reader 2] Waiting to read...
[Writer 1] Waiting to write...
[Reader 1] Reading data: 0
[Reader 2] Reading data: 0
[Writer 1] Writing data...
[Reader 3] Waiting to read...
[Writer 1] Updated data to 1
[Reader 3] Reading data: 1
```